



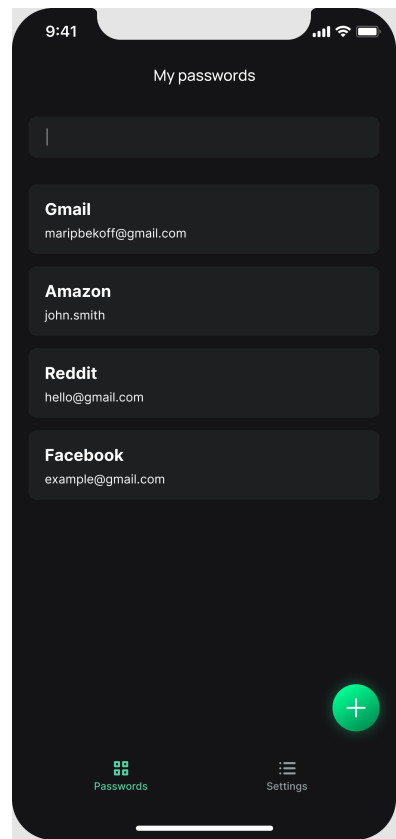
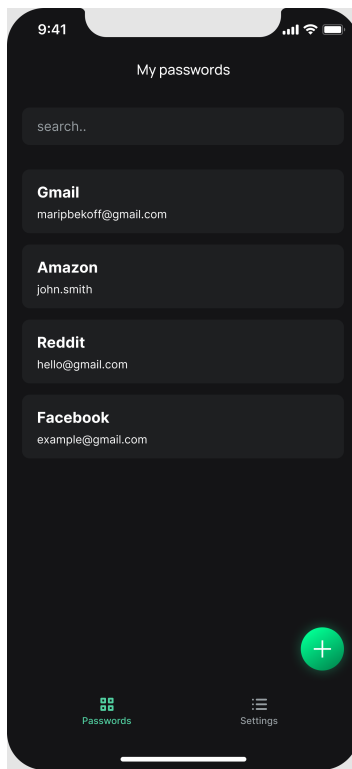
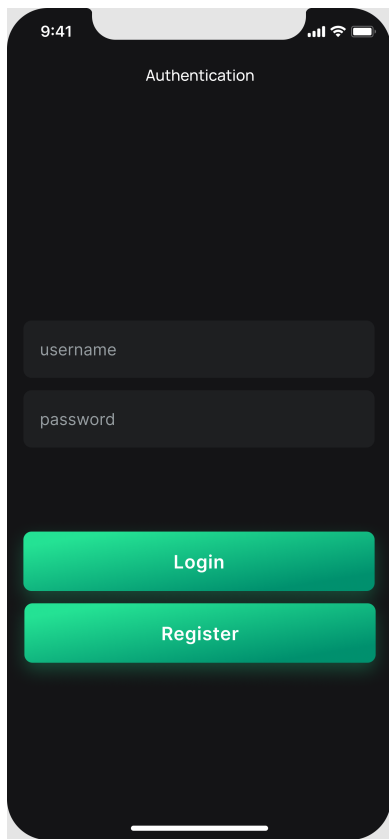
Pocket Vault | Flutter Developer | Eng

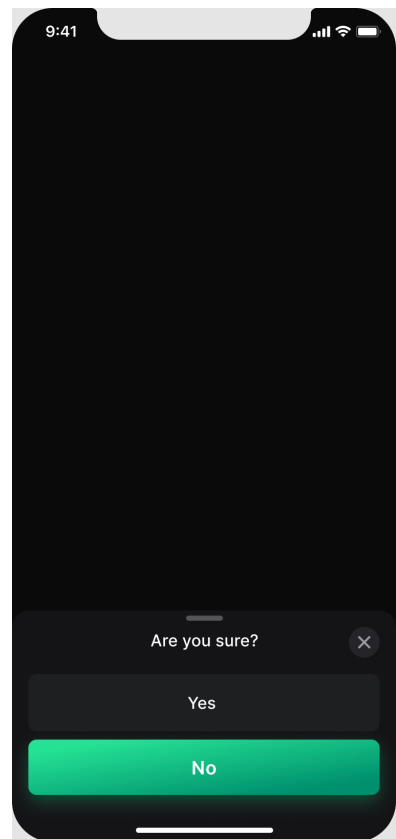
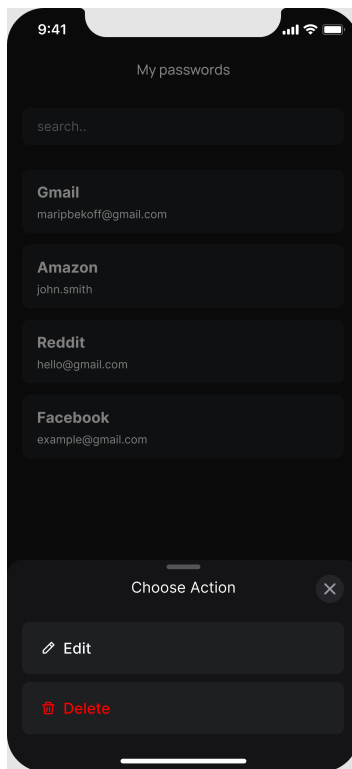
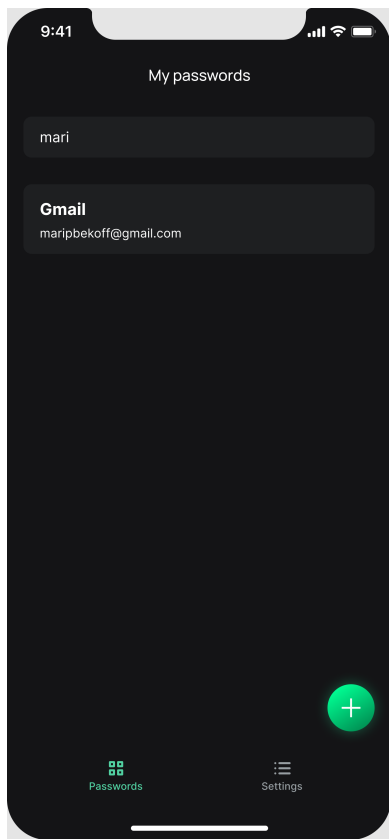
Introduction

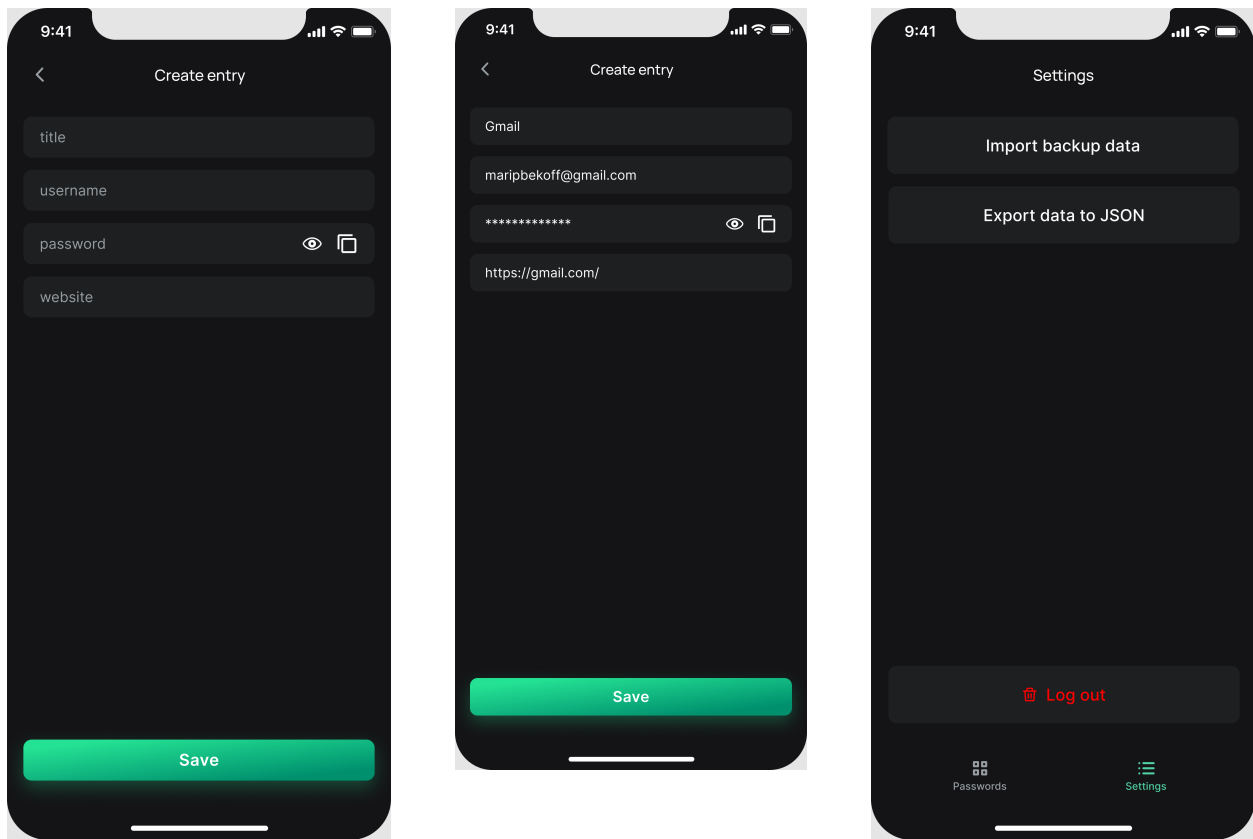
Develop a secure, user-friendly mobile application that functions entirely offline to store sensitive data like passwords, credit card details, and personal notes.

Technical Specifications

- **Languages:** Allow developers to choose their preferred language among Java (Android), Kotlin (Android), Swift (iOS), Objective-C (iOS), or Flutter (cross-platform).
- **Offline Data:** Provide a pre-defined JSON file containing sample entries (passwords, notes, etc.) with relevant attributes (title, username, password, additional notes). The developer will need to parse and store this data locally within the app.
- **No External Libraries:** Require the use of native development tools and libraries only. This ensures developers demonstrate their core mobile development skills without relying on external dependencies.
- **Design:** Share a detailed design mockup of the app's screens (login, main screen, add/edit entry, settings) that developers must meticulously implement. This tests their UI/UX design skills and adherence to the provided visual guidelines.





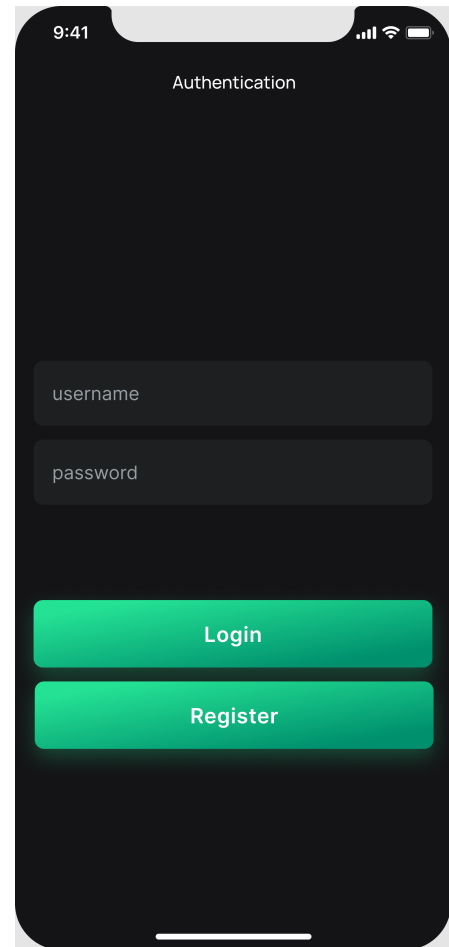


Functionality

- **Security:**
 - Implement robust encryption for stored data using built-in secure storage mechanisms provided by the chosen platform (e.g., Android KeyStore, iOS Keychain).
 - Consider incorporating a feature that allows users to securely back up their data to a local file (optional). This tests the developer's understanding of secure data handling.
- **Animation:** Encourage the use of subtle animations for transitions, button interactions, and other UI elements to enhance user experience. This assesses the developer's proficiency in creating a visually appealing and interactive application.

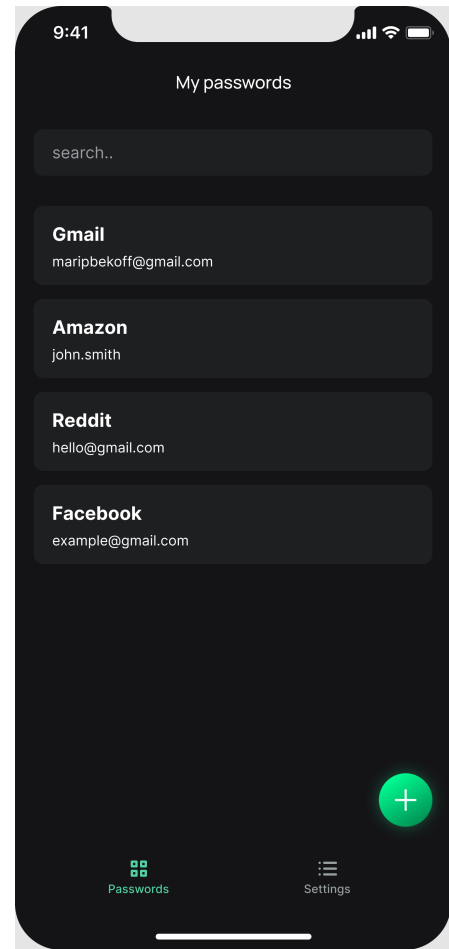
Login Screen

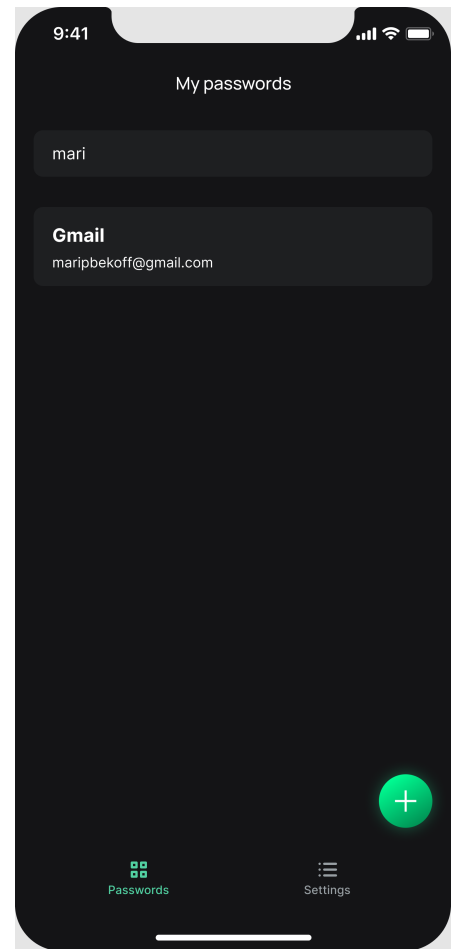
- **Minimalist design:** Emphasize simplicity and ease of use.
- **Secure login options:** Include username/password.
- **Visual feedback:** Provide clear indications for successful login and error messages.
- **Functionality:** Implement a secure login screen with username/password.



Main Screen

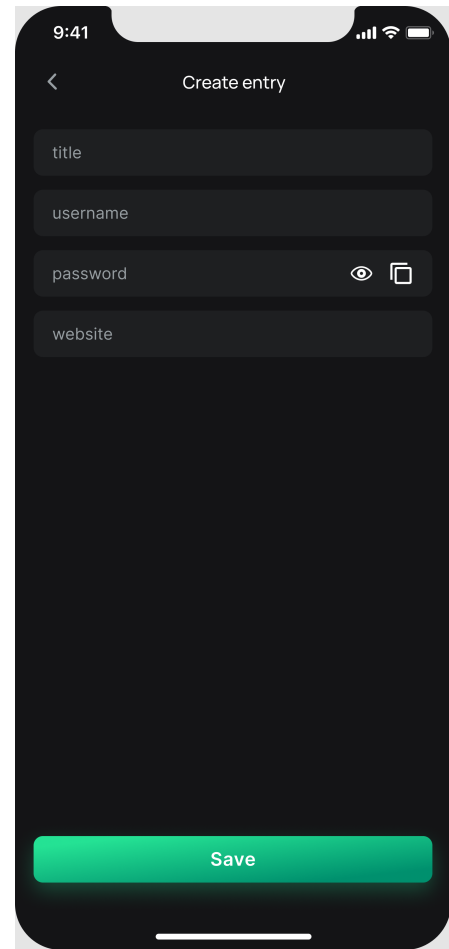
- **List view:** Display a list of existing entries (passwords, notes) in a clear and organized manner.
- **Search bar:** Provide a search bar for quick access to specific entries.
- **Floating action button (FAB):** Add a prominent FAB to easily initiate the process of adding a new entry.
- **Functionality:** Read list of data from JSON

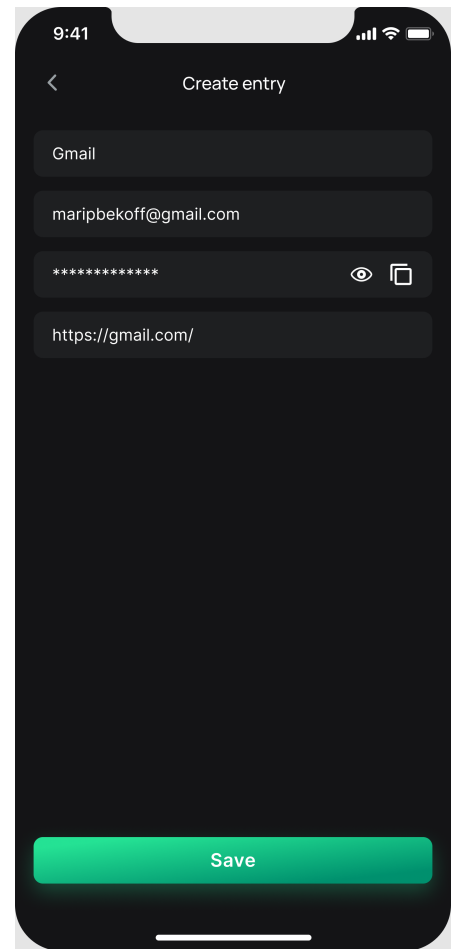




Add/Edit Entry Screen:

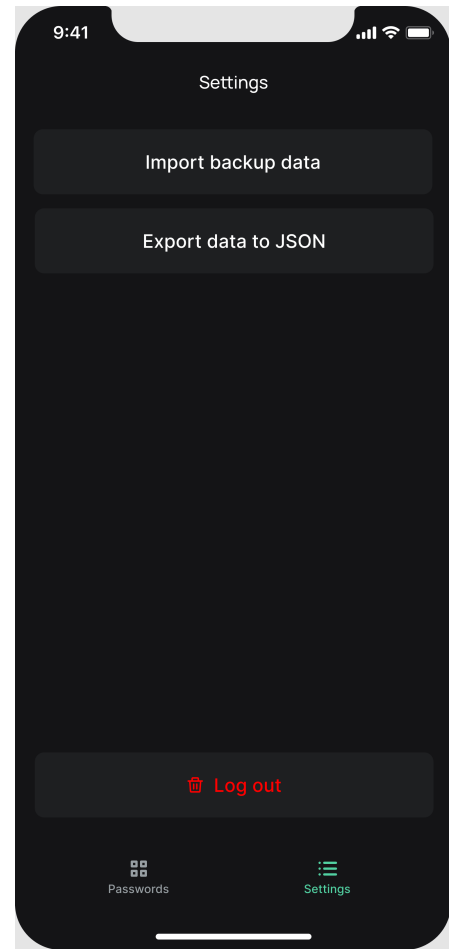
- **Form layout:** Organize input fields in a logical and intuitive manner, using appropriate labels and placeholders.
- **Field types:** Include dedicated fields for different data types, such as password, username, notes, and additional details.
- **Password visibility:** Implement a toggle button to show or hide the entered password for easy verification.
- **Save and cancel buttons:** Provide clear buttons for saving the entry or discarding changes.
- **Functionality:** Allow editing, creating, deleting existing entries.





Settings Screen:

- **Security options:** Allow users to log out account.
- **Backup and restore:** Provide options for creating a local backup of stored data and restoring it from a previous backup file.



Instructions to the Competitor

1. Please create the project with correct package name (Android)/ Organization Identifier (iOS). Naming convention: kz.ws2024.cXX;
2. You should save the project in the folder: XX_Module_KZ;
3. You should rename the generated apk (Android)/ app (iOS) file as XX_Module_KZ.apk (Android)/ XX_Module_KZ.app (iOS), and save it in the root of the XX_Module_KZ folder;
4. The whole XX_Module_KZ folder should be pushed to the desktop;
5. Only works pushed within the specified time will be marked.